

WEB BACKEND CON GO



IL SERVER HTTP INTEGRATO DI GO

Go include una libreria HTTP completa nello standard package **net/http**.

- **http.Handler**: Un'interfaccia usata per gestire le richieste.
- **http.HandleFunc**: Funzione per associare un percorso a una funzione gestore.
- **http.ListenAndServe**: Avvia il server HTTP.
- Il gestore di richieste ha la firma:

```
func handler(w http.ResponseWriter, r *http.Request)
```



ESEMPIO: SERVIZIO "PARI O DISPARI"

Creiamo un endpoint semplice che genera un numero casuale e ne restituisce la parità.

- **`http.ResponseWriter (w)`**: Usato per scrivere la risposta inviata al client (corpo della risposta).
- **`w.Header().Set(...)`**: Utilizzato per impostare gli **header** della risposta (es. Content-Type). **Deve essere chiamato prima di scrivere il corpo!**
- **`fmt.Fprintf(w, ...)`**: Funzione utile che accetta un `io.Writer` (come `w`) per scrivere la risposta.



```
func evenRandomNumber(w http.ResponseWriter, r *http.Request) {  
    // Imposta l'header prima di scrivere il corpo  
    w.Header().Set("Content-Type", "text/plain")  
  
    p := rand.Int()  
    if p % 2 == 0 {  
        fmt.Fprintf(w, "%d is even", p)  
    } else {  
        fmt.Fprintf(w, "%d is odd", p)  
    }  
}
```



LETTURA DEI PARAMETRI: QUERY STRING

Per accettare dati da un client tramite l'URL (metodo GET), si usano i **Query String Parameters**.

Accesso ai Parametri:

1. La richiesta **r** contiene l'URL: **r.URL** (di tipo `*url.URL`).
2. Si usa **.Query()** per ottenere una mappa dei parametri (di tipo `url.Values`).
3. Si usa **.Get("nome_parametro")** per estrarre il valore.



Esempio URL: `http://localhost:8090/?name=John+Doe`

```
func hi(w http.ResponseWriter, r *http.Request) {  
    // Estrae il valore del parametro "name"  
    name := r.URL.Query().Get("name")  
  
    w.Header().Set("Content-Type", "text/plain")  
  
    // Stampa il saluto, utilizzando il nome ricevuto  
    fmt.Fprintf(w, "Hi %s!", name)  
}
```



RICEZIONE DI DATI: POST BODY

Per ricevere dati strutturati (come testo o JSON) inviati con richieste **POST**, è necessario leggere il corpo della richiesta.

Lettura del Body:

- Il corpo della richiesta è accessibile tramite **r.Body** (di tipo `io.ReadCloser`).
- Usiamo **io.ReadAll(r.Body)** per leggere tutto il contenuto in un array di byte (`[]byte`).



Esempio (Dati non strutturati):

```
func hi(w http.ResponseWriter, r *http.Request) {  
    // Legge l'intero corpo della richiesta come []byte  
    body, _ := io.ReadAll(r.Body)  
  
    // Converte il body (si assume sia il nome in plain text) in stringa  
    name := string(body)  
  
    w.Header().Set("Content-Type", "text/plain")  
    fmt.Fprintf(w, "Hi %s!", name)  
  
    // In un server reale: si deve sempre controllare l'errore di io.ReadAll  
    // e l'header Content-Type della richiesta per dati strutturati.  
}
```

Si testa questa funzionalità tipicamente con cURL: `curl -d 'John Doe' http://localhost:8090`



RIASSUNTO E AVVIO

Tutti i gestori devono essere registrati nel *multiplexer* HTTP di Go.

- **Registrazione:** `http.HandleFunc(path, handler_function)`
- **Avvio:** `http.ListenAndServe(":porta", nil)`

```
func main() {  
    // Registra la funzione gestore per il percorso radice  
    http.HandleFunc("/", hi)  
  
    fmt.Println("Starting web server at http://localhost:8090")  
    // Avvia il server sulla porta 8090  
    http.ListenAndServe(":8090", nil)  
}
```



INPUT: *HTTP.REQUEST

Il parametro `r` (di tipo `*http.Request`) contiene tutte le informazioni inviate dal client.



QUERY STRING PARAMETERS (METODO GET)

Questi parametri sono parte dell'URL e vengono usati per passare dati semplici o opzioni di filtraggio (GET `/api/risorse?id=123&sort=asc`).

Componente	Tipo	Descrizione
<code>r.URL.Query()</code>	<code>url.Values</code> (mappa di stringhe)	Una mappa che associa i nomi dei parametri ai loro valori.
Esempio	<pre>id := r.URL.Query().Get("user_id")</pre>	Estrae il valore del parametro <code>user_id</code> .



PATH PARAMETERS

Questi parametri sono integrati nel percorso dell'URL (/users/101).

Componente	Package	Descrizione
Standard Library (net/http)		Il gestore deve analizzare manualmente la stringa <code>r.URL.Path</code> (e.g., usando <code>strings.Split</code>) per estrarre i segmenti variabili. Non raccomandato per API complesse.
Router Esterno (es. Gorilla Mux)	<code>mux.Vars(r)</code>	Restituisce una mappa (<code>map[string]string</code>) contenente i parametri estratti dal path. Questo è l'approccio standard nell'industria Go.
☐ Esempio Manuale	<code>segments := strings.Split(r.URL.Path,</code>	Suddivide il percorso (es. /users/101) in

Componente	Package	Descrizione
	<code>"/"</code>)	segmenti per estrarre l'ID.

LETTURA DEL CORPO DELLA RICHIESTA (POST, PUT)

Per i metodi che inviano dati nel corpo (es. POST o PUT), l'input si trova in `r.Body`.

Componente	Tipo	Descrizione
<code>r.Body</code>	<code>io.ReadCloser</code>	Stream di dati grezzi della richiesta.
<code>io.ReadAll(r.Body)</code>	<code>[]byte, error</code>	Legge l'intero corpo in un array di byte. Fondamentale controllare l'errore!



OUTPUT: HTTP . RESPONSEWRITER

Il parametro `w` (di tipo `http.ResponseWriter`) è l'interfaccia usata per costruire e inviare la risposta al client.



IMPOSTAZIONE DEGLI HEADER

Gli header **devono** essere impostati **prima** di scrivere il corpo della risposta.

Funzione	Descrizione	Utilizzo Tipico
<code>w.Header().Set("Name", "Value")</code>	Imposta un header.	<code>w.Header().Set("Content-Type", "application/json")</code>
<code>w.WriteHeader(code)</code>	Imposta lo status code HTTP. Se non chiamata, Go usa di default 200 OK.	<code>w.WriteHeader(http.StatusCreated)</code> (201).



SCRITTURA DEL CORPO DELLA RISPOSTA

Dopo aver impostato gli header, si scrive il corpo.

Funzione	Descrizione	Utilizzo Tipico
<code>w.Write([]byte)</code>	Scrive un array di byte sul corpo.	Usato per dati grezzi o JSON codificato.
<code>fmt.Fprintf(w, format, args...)</code>	Funzione comoda che scrive direttamente su un <code>io.Writer</code> (come <code>w</code>).	Usato spesso per risposte semplici in testo semplice.



GESTIONE DI DATI STRUTTURATI: JSON

Nel web moderno, JSON è il formato standard per lo scambio di dati. Go usa il package **encoding/json** per mappare i dati tra le struct di Go e le stringhe JSON.



LEGGERE JSON DAL CORPO DELLA RICHIESTA (DECODING)

Per leggere un corpo JSON in una struct Go, si usa `json.NewDecoder`.

```
type UserInput struct {  
    Name string `json:"name"`  
    Age  int   `json:"age"`  
}  
// ...  
var input UserInput  
err := json.NewDecoder(r.Body).Decode(&input)  
// ...
```



SCRIVERE JSON NELLA RISPOSTA (ENCODING)

Per inviare una struct Go come risposta JSON, si usa `json.Marshal` o `json.NewEncoder`.

```
type UserResponse struct {  
    Message string `json:"status"`  
}  
// ...  
response := UserResponse{Message: "Success"}  
w.Header().Set("Content-Type", "application/json")  
json.NewEncoder(w).Encode(response)
```



ESERCIZIO: BACKEND API PER "HI-LO"



SPECIFICHE API (REQUISITI MINIMI)

Definiamo le rotte (endpoint) che il server deve esporre.

Metodo	Percorso	Azione	Risposta (JSON)
POST	/games	Avvia una nuova partita.	<code>{"game_id": "ID_UNIVOCO_GIOCO"}</code>
POST	/games/{id}/guess	Invia un tentativo per una partita specifica.	<code>{"result": "hi"} (o "lo", "correct")</code>
GET	/games/{id}	(Bonus) Mostra lo stato attuale della partita.	<code>{"guesses_made": 5, "status": "in_progress"}</code>



SUGGERIMENTI PACCHETTI

- `net/http`: Per il server e i gestori (handler).
- `encoding/json`: Per `NewDecoder` (leggere JSON) e `NewEncoder` (scrivere JSON).
- `math/rand`: Per generare il numero segreto all'avvio della partita.
- `time`: Per inizializzare `rand.Seed()`.
- `sync`: Per il Mutex.



ESERCIZIO: HANDLENEWGAME (POST /GAMES)

1. Generare un numero segreto (es. `rand.Intn(100) + 1`).
2. Generare un ID univoco per la partita (es. un contatore o `fmt.Sprintf`).
3. Creare la nuova `GameState`.
4. **Bloccare il Mutex** (`mu.Lock()`).
5. Salvare la nuova partita nella mappa `games`.
6. **Sbloccare il Mutex** (`mu.Unlock()`).
7. Rispondere al client con il JSON contenente il `game_id`.



ESERCIZIO: HANDLEGUESS (POST /GAMES/{ID}/GUESS)

Cosa deve fare il vostro gestore:

1. Ottenere `{id}` dal percorso (usando `mux.Vars(r)`).
2. Leggere il JSON dal corpo (`r.Body`) per ottenere il tentativo del client.
3. **Bloccare il Mutex** (o `mu.RLock()` se si usa `RWMutex`).
4. Recuperare lo stato della partita (`games[id]`).
5. **Sbloccare il Mutex** (o `mu.RUnlock()`).
6. Controllare se lo stato esiste (errore 404 se non esiste).
7. Confrontare il tentativo con `SecretNumber` e definire la risposta ("hi", "lo", "correct").
8. (Opzionale: aggiornare lo stato con il nuovo tentativo, richiederà `mu.Lock()`).
- ☐ 9. Rispondere al client con il JSON contenente il risultato.

PRONTI?

Questo esercizio testa:

- Gestione di `http.Request` e `http.ResponseWriter`.
- Routing (con mux).
- Marshalling e Unmarshalling JSON.
- Gestione dello stato in memoria.
- Sincronizzazione base (Mutex).

